

در این بخش ابتدا فایل دیتاست موردنظر که به صورت فرمت JSON است می خوانیم .

با دستور معمول پای اسپارک برای خواندن فایل ها سعی کردیم فایل دیتاست را بخوانیم که با خطای زیر مواجه شدیم :

```
[3] import pyspark
    from pyspark.sql import SparkSession

    # ایجاد یک SparkSession
    spark = SparkSession.builder.appName("ReadJSONExample").getOrCreate()

    # خواندن فایل JSON
    df = spark.read.json("/content/gdrive/MyDrive/ingredients/train.json")

    # نمایش DataFrame
    df.show()
```

AnalysisException Traceback (most recent call last)
<ipython-input-3-d30e928730b8> in <cell line: 11>()
 9
 10 # نمایش DataFrame
----> 11 df.show()

----- 3 frames -----
/usr/local/lib/python3.10/dist-packages/pyspark/errors/exceptions/captured.py in deco(*a, **kw)
 183 # Hide where the exception came from that shows a non-Pythonic
 184 # JVM exception message.
--> 185 raise converted from None
 186 else:
 187 raise

AnalysisException: Since Spark 2.3, the queries from raw JSON/CSV files are disallowed when the
referenced columns only include the internal corrupt record column
(named _corrupt_record by default). For example:
spark.read.schema(schema).csv(file).filter(\$"_corrupt_record".isNotNull).count()
and spark.read.schema(schema).csv(file).select("_corrupt_record").show().
Instead, you can cache or save the parsed results and then send the same query.
For example, val df = spark.read.schema(schema).csv(file).cache() and then
df.filter(\$"_corrupt_record".isNotNull).count().

این خطا به دلیل عدم مطابقت ساختار JSON در فایل با انتظارات PySpark رخ می دهد. معمولاً وقتی داده های JSON شما به درستی قالب بندی نشده باشند (مثلاً هر سطر فایل JSON یک شیء JSON مجزا نباشد) PySpark نمی تواند آن ها را به درستی بخواند و ستون `_corrupt_record` را تولید می کند تا نشان دهد داده ها مشکل دار هستند.

برای رفع این خطا از دستور زیر استفاده کردیم :

```
[26] # Read multiline json file
      multiline_df = spark.read.option("multiline", "true") \
          .json("/content/gdrive/MyDrive/ingredients/train.json")
      multiline_df.toPandas()
```

با استفاده از این دستور ، فایل train.json خوانده شد که نمونه ای از این داده ها به صورت زیر است :

index	cuisine	id	ingredients
0	greek	10259	romaine lettuce,black olives,grape tomatoes,garlic,pepper,purple onion,seasoning,garbanzo beans,feta cheese crumbles
1	southern_us	25693	plain flour,ground pepper,salt,tomatoes,ground black pepper,thyme,eggs,green tomatoes,yellow corn meal,milk,vegetable oil
2	filipino	20130	eggs,pepper,salt,mayonaise,cooking oil,green chilies,grilled chicken breasts,garlic powder,yellow onion,soy sauce,butter,chicken livers
3	indian	22213	water,vegetable oil,wheat,salt
4	indian	13162	black pepper,shallots,comflour,cayenne pepper,onions,garlic paste,milk,butter,salt,lemon juice,water,chili powder,passata,oil,ground cumin,boneless chicken skinless thigh,garam masala,double cream,natural yogurt,bay leaf
5	jamaican	6602	plain flour,sugar,butter,eggs,fresh ginger,root,salt,ground cinnamon,milk,vanilla extract,ground ginger,powdered sugar,baking powder
6	spanish	42775	olive oil,salt,medium shrimp,pepper,garlic,chopped cilantro,jalapeno chilies,flat leaf parsley,skirt steak,white vinegar,sea salt,bay leaf,chorizo sausage
7	italian	3735	sugar,pistachio nuts,white almond bark,flour,vanilla extract,olive oil,almond extract,eggs,baking powder,dried cranberries
8	mexican	16903	olive oil,purple onion,fresh pineapple,pork,pobiano peppers,com tortillas,cheddar cheese,ground black pepper,salt,iceberg lettuce,lime,jalapeno chilies,chopped cilantro,fresh
9	italian	12734	chopped tomatoes,fresh basil,garlic,extra-virgin olive oil,kosher salt,flat leaf parsley
10	italian	5875	pimentos,sweet pepper,dried oregano,olive oil,garlic,sharp cheddar cheese,pepper,swiss cheese,provolone cheese,canola oil,mushrooms,black olives,sausages
11	chinese	45887	low sodium soy sauce,fresh ginger,dry mustard,green beans,white pepper,sesame oil,scallions,canola oil,sugar,Shaoxing wine,garlic,ground turkey,water,crushed red pepper flakes,corn starch
12	italian	2698	Italian parsley,leaves,walnuts,hot red pepper flakes,extra-virgin olive oil,fresh lemon juice, trout fillet,garlic cloves,chipotle chile,fine sea salt,flat leaf parsley
13	mexican	41995	ground cinnamon,fresh cilantro,chili powder,ground coriander,kosher salt,ground black pepper,garlic,plum tomatoes,avocado,lime juice,flank steak,salt,ground cumin,black pepper,olive oil,crushed red pepper flakes,onions
14	italian	31908	fresh parmesan cheese,butter,all-purpose flour,fat free less sodium chicken broth,chopped fresh chives,gruyere cheese,ground black pepper,bacon slices,gnocchi,fat free milk,cooking spray,salt
15	indian	24717	tumeric,vegetable stock,tomatoes,garam masala,naan,red lentils,red chili peppers,onions,spinach,sweet potatoes
16	british	34466	greek yogurt,lemon curd,confectioners sugar,raspberries
17	italian	1420	italian seasoning,broiler-fryer chicken,mayonaise,zesty italian dressing
18	thai	2941	sugar,hot chili,asian fish sauce,lime juice
19	vietnamese	8152	soy sauce,vegetable oil,red bell pepper,chicken broth,yellow squash,garlic chili sauce,sliced green onions,broccolini,salt,fresh lime juice,cooked rice,chicken breasts,corn starch
20	thai	13121	pork loin,roasted peanuts,chopped cilantro,fresh hoisin sauce,creamy peanut butter,chopped fresh mint,thai basil,rice,medium shrimp,water,rice noodles,beansprouts
21	mexican	40523	roma tomatoes,kosher salt,purple onion,jalapeno chilies,lime,chopped cilantro
22	southern_us	40989	low-fat mayonnaise,pepper,salt,baking potatoes,eggs,spicy brown mustard
23	chinese	29630	sesame seeds,red pepper,yellow peppers,water,extra firm tofu,broccoli,soy sauce,orange bell pepper,amrroot powder,fresh ginger,sesame oil,red curry paste
24	italian	49136	marinara sauce,flat leaf parsley,olive oil,linguine,capers,crushed red pepper flakes,olives,lemon zest,garlic

Show 25 per page

1210100700760800

در ادامه پیش پردازش های لازم برای شروع کار را انجام می دهیم که عبارت است از :

- تبدیل حروف مواد غذایی به حروف کوچک (lower case کردن) که با قطعه کد زیر این کار را انجام دادیم :

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, expr

for ingredients in multiline_df.columns:
    # بررسی اینکه آیا نوع ستون آرایه از رشته است یا نه
    if dict(multiline_df.dtypes)[ingredients] == "array<string>":
        # تبدیل عناصر آرایه به حروف کوچک
        multiline_df = multiline_df.withColumn(ingredients, expr(f"transform({ingredients}, x -> lower(x))"))
    elif dict(multiline_df.dtypes)[ingredients] == "string":
        # تبدیل رشته های تکی به حروف کوچک
        multiline_df = multiline_df.withColumn(ingredients, lower(col(ingredients)))
```

- حذف s/es انتهایی کلمات و تبدیل آنها به کلمه ی مفرد با قطعه کد زیر :

```
[ ] from pyspark.sql import SparkSession
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType, ArrayType
import inflect

# Initialize inflect engine
p = inflect.engine()

def singularize(word):
    if isinstance(word, list):
        # If word is a list, singularize each element
        return [p.singular_noun(w) if p.singular_noun(w) else w for w in word]
    elif isinstance(word, str):
        # If word is a string, singularize it directly
        return p.singular_noun(word) if p.singular_noun(word) else word
    return word

# Register the UDF with PySpark, specifying ArrayType for list inputs
singularize_udf = udf(singularize, ArrayType(StringType()))

# Apply the UDF to the specific columns (replace 'column_name' with your actual column)
df_singular = multiline_df.withColumn("ingredients", singularize_udf(multiline_df["ingredients"]))

df_singular.toPandas()
```

که کلمات جمع (به عنوان مثال tomatoes , black olives , garbanzo beans) به شکل زیر :

index	cuisine	id	ingredients
0	greek	10259	romaine lettuce,black olives,grape tomatoes,garlic,pepper,purple onion,seasoning,garbanzo beans,feta cheese crumbles

به کلمات مفرد به صورت زیر تبدیل می شوند :

index	cuisine	id	ingredients
0	greek	10259	romaine lettuce,black olive,grape tomato,garlic,pepper,purple onion,seasoning,garbanzo bean,feta cheese crumble

یک نمای کلی از نمونه ای از دیتاست بعد از انجام این عمل به صورت زیر است :

index	cuisine	id	ingredients
0	greek	10259	romaine lettuce,black olive,grape tomato,garlic,pepper,purple onion,seasoning,garbanzo bean,feta cheese crumble
1	southern_us	25693	plain flour,ground pepper,salt,tomato,ground black pepper,thyme,egg,green tomato,yellow corn meal,vegetable oil
2	filipino	20130	egg,pepper,salt,mayonaise,cooking oil,green chili,grilled chicken breast,garlic powder,yellow onion,soy sauce,butter,chicken liver
3	indian	22213	water,vegetable oil,wheat,salt
4	indian	13162	black pepper,shallot,cornflour,cayenne pepper,onion,garlic paste,milk,butter,salt,lemon juice,water,chili powder,passata,oil,ground cumin,boneless chicken skinless thigh,garam masala,double cream,natural yogurt,bay leaf
5	jamaican	6602	plain flour,sugar,butter,egg,fresh ginger root,salt,ground cinnamon,milk,vanilla extract,ground ginger,powdered sugar,baking powder
6	spanish	42779	olive oil,salt,medium shrimp,pepper,garlic,chopped cilantro,jalapeno chili,flat leaf parsley,skirt steak,white vinegar,sea salt,bay leaf,chorizo sausage
7	italian	3735	sugar,pistachio nut,white almond bark,flour,vanilla extract,olive oil,almond extract,egg,baking powder,dried cranberry
8	mexican	16903	olive oil,purple onion,fresh pineapple,pork,poblano pepper,corn tortilla,cheddar cheese,ground black pepper,salt,iceberg lettuce,lime,jalapeno chili,chopped cilantro fresh
9	italian	12734	chopped tomato,fresh basil,garlic,extra-virgin olive oil,kosher salt,flat leaf parsley
10	italian	5875	pimento,sweet pepper,dried oregano,olive oil,garlic,sharp cheddar cheese,pepper,swiss cheese,provone cheese,canola oil,mushroom,black olive,sausage
11	chinese	45887	low sodium soy sauce,fresh ginger,dry mustard,green bean,white pepper,sesame oil,scallion,canola oil,sugar,shaoxing wine,garlic,ground turkey,water,crushed red pepper flake,corn starch
12	italian	2698	italian parsley leaf,walnut,hot red pepper flake,extra-virgin olive oil,fresh lemon juice,trout fillet,garlic clove,chipotle chile,fine sea salt,flat leaf parsley
13	mexican	41995	ground cinnamon,fresh cilantro,chili powder,ground coriander,kosher salt,ground black pepper,garlic,plum tomato,avocado,lime juice,flank steak,salt,ground cumin,black pepper,olive oil,crushed red pepper flake,onion
14	italian	31908	fresh parmesan cheese,butter,all-purpose flour,fat free less sodium chicken broth,chopped fresh chive,gruyere cheese,ground black pepper,bacon slice,gnocchi,fat free milk,cooking spray,salt
15	indian	24717	tumeric,vegetable stock,tomato,garam masala,naan,red lentil,red chili pepper,onion,spinach,sweet potato
16	british	34466	greek yogurt,lemon curd,confectioners sugar,raspberry
17	italian	1420	italian seasoning,broiler-fryer chicken,mayonaise,zesty italian dressing
18	thai	2941	sugar,hot chili,asian fish sauce,lime juice
19	vietnamese	8152	soy sauce,vegetable oil,red bell pepper,chicken broth,yellow squash,garlic chili sauce,sliced green onion,broccolini,salt,fresh lime juice,cooked rice,chicken breast,corn starch
20	thai	13121	pork loin,roasted peanut,chopped cilantro fresh,hoisin sauce,creamy peanut butter,chopped fresh mint,thai basil,rice,medium shrimp,water,rice noodle,beansprout
21	mexican	40523	roma tomato,kosher salt,purple onion,jalapeno chili,lime,chopped cilantro
22	southern_us	40989	low-fat mayonnaise,pepper,salt,baking potato,egg,spicy brown mustard
23	chinese	29630	sesame seed,red pepper,yellow pepper,water,extra firm tofu,broccoli,soy sauce,orange bell pepper,arrowroot powder,fresh ginger,sesame oil,red curry paste
24	italian	49136	marinara sauce,flat leaf parsley,olive oil,linguine,caper,crushed red pepper flake,olive,lemon zest,garlic

Show 25 per page

– حذف stopwords ها با قطعه کد زیر :

```
[5] from pyspark.ml.feature import StopWordsRemover

# 2. حذف Stop Words
remover = StopWordsRemover(inputCol="ingredients", outputCol="StopWordsRemoved")
filtered_df = remover.transform(df_singular)
```

در قسمت دوم می‌خواهیم که بردارهای one-hot متناظر با هر ماده‌ی غذایی را ایجاد کنیم. برای این کار ابتدا تمام مواد غذایی را استخراج و جدا می‌کنیم و به هر رشته یک اندیس عددی اختصاص می‌دهیم:

```
from pyspark.ml.feature import StringIndexer
from pyspark.sql.functions import explode

df_exploded = filtered_df.withColumn("ingredient", explode("StopWordsRemoved"))

indexer = StringIndexer(inputCol="ingredient", outputCol="ingredientIndex")
indexed = indexer.fit(df_exploded).transform(df_exploded)
indexed.show()
```

cuisine	id	ingredients	StopWordsRemoved	ingredient	ingredientIndex
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	romaine lettuce	296.0
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	black olive	337.0
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	grape tomato	341.0
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	garlic	4.0
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	pepper	10.0
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	purple onion	27.0
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	seasoning	518.0
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	garbanzo bean	491.0
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	feta cheese crumble	216.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	plain flour	480.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	ground pepper	200.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	salt	0.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	tomato	16.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	ground black pepper	8.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	thyme	215.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	egg	12.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	green tomato	604.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	yellow corn meal	239.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	milk	23.0
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	vegetable oil	11.0

only showing top 20 rows

و بعد با استفاده از تابع OneHotEncoder بردارهای متناظر هر کدام از مواد غذایی را به دست می‌آوریم:

```
[9] from pyspark.ml.feature import OneHotEncoder

encoder = OneHotEncoder(inputCol="ingredientIndex", outputCol="ingredientVec")
df_one_hot = encoder.fit(indexed).transform(indexed)

df_one_hot.show()
```

cuisine	id	ingredients	StopWordsRemoved	ingredient	ingredientIndex	ingredientVec
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	romaine lettuce	296.0	(6694,[296],[1.0])
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	black olive	337.0	(6694,[337],[1.0])
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	grape tomato	341.0	(6694,[341],[1.0])
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	garlic	4.0	(6694,[4],[1.0])
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	pepper	10.0	(6694,[10],[1.0])
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	purple onion	27.0	(6694,[27],[1.0])
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	seasoning	518.0	(6694,[518],[1.0])
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	garbanzo bean	491.0	(6694,[491],[1.0])
greek	10259	[romaine lettuce,...]	[romaine lettuce,...]	feta cheese crumble	216.0	(6694,[216],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	plain flour	480.0	(6694,[480],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	ground pepper	200.0	(6694,[200],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	salt	0.0	(6694,[0],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	tomato	16.0	(6694,[16],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	ground black pepper	8.0	(6694,[8],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	thyme	215.0	(6694,[215],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	egg	12.0	(6694,[12],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	green tomato	604.0	(6694,[604],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	yellow corn meal	239.0	(6694,[239],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	milk	23.0	(6694,[23],[1.0])
southern_us	25693	[plain flour, gro...]	[plain flour, gro...]	vegetable oil	11.0	(6694,[11],[1.0])

only showing top 20 rows

حالا این بردارهایی که برای هر کلمه به دست آمده را در یک ماده ی غذایی جمع می کنیم و برای هر کدام یک representation به صورت زیر ایجاد می کنیم :

```

from pyspark.ml.linalg import Vectors, DenseVector
from pyspark.sql.functions import col, udf, collect_list
from pyspark.ml.linalg import VectorUDT
from pyspark.sql.types import ArrayType, FloatType

# مرحله 1: جمع آوری بردارها برای هر id
grouped_df = df_one_hot.groupBy("id").agg(
    collect_list("ingredientVec").alias("vectors")
)

# مرحله 2: ترکیب بردارها به یک بردار واحد
def merge_vectors(vectors):
    merged = sum(vectors, DenseVector([0.0] * len(vectors[0]))) # بردار نهایی
    return merged

merge_udf = udf(merge_vectors, VectorUDT())

# افزودن ستون ترکیب شده
result_df = grouped_df.withColumn("combined_vector", merge_udf(col("vectors")))

# نمایش نتیجه
result_df.select("id", "combined_vector").show(truncate=False)

```

[illegible]

با استفاده از این روش بردار ها را ادغام کردیم اما هنگام پردازش الگوریتم k-means ، با مشکل زمان طولانیه پردازش مواجه شدیم که تصمیم گرفتیم از SparseVector برای ادغام بردار ها استفاده کنیم که به صورت زیر آن را نوشتیم :

```
[ ]
from pyspark.ml.linalg import SparseVector, VectorUDT
from pyspark.sql.functions import col, udf, collect_list
from pyspark.sql.types import IntegerType

# مرحله 1: جمع آوری بردارها برای هر id
grouped_df = df_one_hot.groupBy("id", "cuisine").agg(
    collect_list("ingredientVec").alias("vectors")
)

# مرحله 2: ترکیب بردارها با یک SparseVector
def merge_sparse_vectors(vectors):
    size = len(vectors[0]) # اندازه بردار
    indices = []
    values = []

    for vec in vectors:
        sparse_vec = vec.toArray() if isinstance(vec, SparseVector) else vec
        for i, value in enumerate(sparse_vec):
            if value != 0.0:
                indices.append(i)
                values.append(value)

    # حذف مقادیر تکراری و جمع مقادیر مشابه
    combined_indices = list(set(indices))
    combined_indices.sort()
    combined_values = [sum(values[j] for j, idx in enumerate(indices) if idx == i) for i in combined_indices]

    return SparseVector(size, combined_indices, combined_values)

# تعریف UDF با استفاده از VectorUDT
merge_udf = udf(merge_sparse_vectors, VectorUDT())

# افزودن ستون ترکیبی
result_df = grouped_df.withColumn("combined_vector", merge_udf(col("vectors")))

# نمایش نتیجه
# result_df.select("id", "cuisine", "combined vector").show(truncate=False)
```

همین کار را به کمک فضای برداری word2vec تکرار می کنیم و برای هر مجموعه مواد غذایی یک representation ایجاد می کنیم . تو این قطعه کد برای هر کلمه از ۱۰۰ بردار برای افزایش دقت استفاده شده است :

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import Word2Vec

# Word2Vec تنظیمات
word2vec = Word2Vec(vectorSize=300, minCount=0, inputCol="StopWordsRemoved", outputCol="resultvector")

# Word2Vec مدل
model = word2vec.fit(filtered_df)

# Word2Vec تبدیل دیتا به فضای
result = model.transform(filtered_df)

# نمایش نتایج
result.select("id", "resultvector").show(truncate=False)
```

id	resultvector
18259	[0.0081974748428911,-0.002956093211347858,-0.008081158312658468,-0.03634041941008355,-0.06288737162119812,0.028527929809772305,-0.067656250722924,0.009584514165504111,0.03116623850332366,0.00988986779954698,-0.04433671136697133,0.0580306425693]
25693	[0.002809947356581688,-0.9359046329151515,-0.02128759183158929,0.08507240360433406,0.004587325860153545,-0.03642460356720469,0.03638398595615178,0.031896387988870796,0.006059687542305752,0.01355296377600594,-0.04306469307365743,-0.04996628130]
28130	[0.00870154783478938,-0.02627536068515231,-0.037306562395921596,0.06332588083461678,0.04233785958661933,0.021644707846765716,0.003863840460932503,0.029012900854771335,0.08344186982139945,0.04250013418883706,-0.041984690236859024,-0.0563422213]
12213	[0.0545408390462395,-0.03623909702527595,0.01674699260245604,0.02060475912934034,0.02471570181660354,0.02323674294166267,-0.016293991007008674,-0.02957057044841349,0.07077697734348476,-0.02283529019607231,0.004882750101387501,-0.0090813162]
13162	[0.07739757983945311,-0.0164814192103222,-0.03191975861554965,0.03515352014801465,0.004397431178949774,-0.041254665423184636,0.007236664730589837,0.005750476551475004,0.06321749358903617,0.060804089484736326,0.009889352921136663,-0.057166602]
6602	[0.009127309119018415,-0.03753180660229797,0.039521904608894445,0.12166079723586638,-0.015939651833226282,-0.01009056530892849,0.07618694426481857,0.029050250692913927,-0.00925046302533398,-0.002757726181359973,-0.04442879434403342,-0.142779]
42779	[0.048627762679115391,0.03863302808111677,-0.020464926456602722,0.0012333130893798976,0.040463445314134545,-0.021458694711327553,-0.01802380049887758,0.011496830373429336,0.020993917983455155,-0.02988481206389574,-0.010653724822287377,-0.013735]
3735	[0.03415445806442983,-0.0040625864458070337,0.04936629081785213,0.12262233071087394,-0.010426492174156012,-0.011471785604953766,0.0740446448028076,0.03315671796444804,-0.049961750337388366,-0.019566768198274076,-0.08057477202964947,-0.1316903]
16903	[0.1063637390302924,0.02035438360910223,-0.014599691145122051,-0.031447685502756104,0.07438989664213017,0.013306075272726793,-0.03053725521636074,0.02739986730130315,0.04949257648191773,-0.029020940473425276,-0.033144814893603325,0.03512734]
12734	[0.07212462165392953,0.023340590670999207,0.024608359611213207,0.010004513578469423,0.028150134176636733,0.00495456562040453,-0.04068432391310731,0.024984935910443764,-0.008386782680940574,-0.013002637618531782,-0.05956280728181203,0.0115875]
5875	[0.036034143870123304,0.03227521230180103,-0.01383517381663506,0.038141690409527376,0.04668740433855699,-0.009175728173711553,-0.008834464976993892,0.006072159207318552,0.024950754255629506,0.037800750435473256,-0.04844089606418633,-0.0145887]
45887	[0.0032145775854587554,-0.035003595087133965,-0.04100242054943616,0.06201242556174596,0.017318873448918262,0.10336620689680179,-0.0451619158188502,0.03226124111097306,0.012896742795900477,0.06748856655322015,-0.05309621032016973,-0.09842698]
2698	[0.0504149222122133,0.028950020042248073,0.040885016623360575,2.1599305327981713E-4,0.009544457634910943,0.008812345284968616,-0.0307253310456872,-0.014969195961020887,-0.07843962165061386,0.0035120678599923853,-0.031337936082854866,0.0141995]
41995	[0.09117792278309078,0.015591247108258197,-0.01828499633462771,-0.024568868143593956,0.03435706906020641,-0.0011911648599540485,-0.04132126756560277,0.023378184174790182,0.04207081553142737,0.024915130644598427,-0.011208619100644306,0.0031908]
31908	[0.01910304728274544,0.05609628274651348,0.06436820492806503,0.052509247161340656,-0.0375957627395959,-0.04907078125203649,0.03410693444311619,0.03068036485376705,-0.05645614257082343,0.01844403537680777,-0.05900640797118344,-0.0247924717]
24717	[0.1123585632652045,-0.03970844596624375,-0.034816223183553055,0.0019530018791556359,-0.0394920913502574,-0.0614493815228433,0.0016076842497568578,0.024949846137315036,0.07055677869357169,0.08712658425793052,0.02709480926860124,-0.0670434466]
34466	[0.010781605262309313,-0.019625780318165198,0.050558828834883869,0.07525669730966911,0.0016496091266162694,7.097185589373112E-4,0.014605822652811185,0.0466434576227702,-0.06950694043189287,-0.032437501940876245,-0.037551917775999755,-0.11420]
1420	[0.016646098022647202,0.03109850356122479,-0.011087247519753873,0.0691143162548542,0.06382788997143507,0.008976395998615772,0.037454629899900261,0.008909975178539753,0.02276072360109538,0.012729537207633257,-0.05102825892390683,-0.006831212941]
2941	[0.08289919258095324,-0.05447908560745418,0.012007351571810432,0.013624736093333922,0.008191443979740143,0.057048684218898416,-0.00956360436975956,0.027078290295321494,0.03412946389289573,-0.0210268060689744353,-0.011964502977207303,-0.0718152]
6152	[0.03471565801793566,-0.024008080563559246,-0.02057383581995964,0.0186276262142542,0.03348370427444864,0.07173345381250748,-0.03517459286376834,0.0055325556761370255,0.04168001143079423,0.05553467806952254,-0.026560006010441457,-0.0458161625693]

این representation را به کمک بردار میانگین هم انجام دادیم :

```
[16] from pyspark.ml.linalg import Vectors
from pyspark.sql import functions as F
from pyspark.sql.types import ArrayType, DoubleType

# Vector به Array تبدیل
result = result.withColumn("result_array", F.udf(lambda v: v.toArray().toList(), ArrayType(DoubleType()))(F.col("resultvector")))

# محاسبه میانگین برای هر بردار
df_avg = result.select(
    "id",
    F.expr("aggregate(result_array, 0D, (acc, x) -> acc + x, acc -> acc / size(result_array))").alias("result_avg")
)
```

index	id	result_avg
0	10259	0.0026219206832311767
1	25693	0.0003679121204803589
2	20130	0.0006096136608842561
3	22213	5.1312449180223480e-05
4	13162	0.001485890432000695
5	6602	0.00033164657371041
6	42779	0.001700366470861179
7	3735	-0.00030694436236778185
8	16903	0.003700931274046265
9	12734	0.0015260915752272305

در مرحله ی بعد شروع می کنیم به خوشه بندی با استفاده از الگوریتم k-means . این کار را ابتدا با استفاده از بردار های one-hot ادغام شده به صورت زیر انجام دادیم :

```
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator
import matplotlib.pyplot as plt

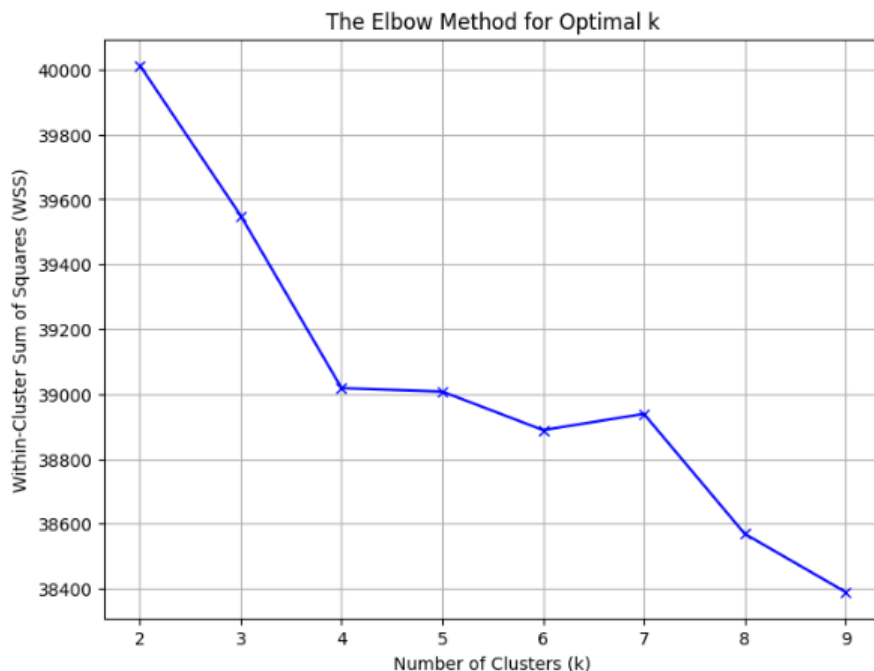
cost = []
K = range(2, 10) # بررسی k از 1 تا 9

for k in K:
    kmeans = KMeans().setK(k).setSeed(1).setFeaturesCol("combined_vector")
    model = kmeans.fit(sample_data)
    wss = model.summary.trainingCost # مجموع مربعات خطا
    cost.append(wss)

# پیشبینی خوشه ها
predictions = model.transform(sample_data)

# رسم نمودار Elbow
plt.figure(figsize=(8, 6))
plt.plot(K, cost, 'bx-')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Within-Cluster Sum of Squares (WSS)')
plt.title('The Elbow Method for Optimal k')
plt.grid()
plt.show()
```

نمودار elbow به دست آمده در زیر قابل مشاهده است :



به کمک بردار ویژگی میانگین word2vec نیز الگوریتم k-means را اجرا کردیم که قطعه کد آن به صورت زیر است :

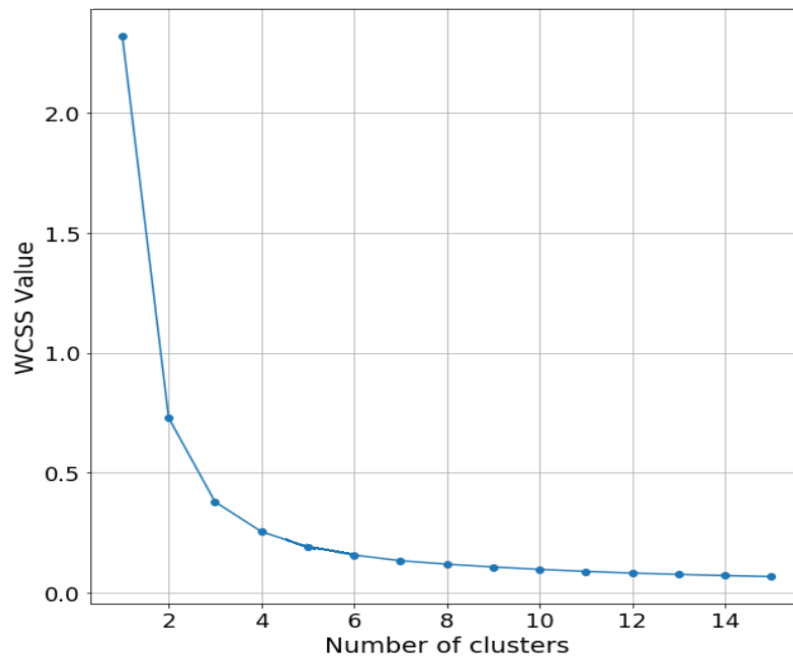
```
[ ] import matplotlib.pyplot as plt
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator

# محاسبه خطای WCSS
cost = []
k_values = range(2, 15)

for k in k_values:
    print(f"Running KMeans with k={k}")
    kmeans = KMeans().setK(k).setSeed(1).setFeaturesCol("features")
    model = kmeans.fit(sample_data)
    wsse = model.summary.trainingCost # خطای WSSSE
    cost.append(wsse)

# رسم نمودار elbow
plt.figure(figsize=(8, 6))
plt.plot(k_values, cost, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS Value")
plt.show()
```

و نمودار elbow به دست آمده به شکل زیر است :



در زیر نمونه ای از پیش بینی خوشه های الگوریتم k-means را مشاهده می کنید :

id	cuisine	vectors	combined_vector	prediction
15	mexican	[(6694,[3],[1.0])...]	(6694,[3,43,57,80...]	2
46	indian	[(6694,[11],[1.0])...]	(6694,[0,2,11,46,...]	8
54	thai	[(6694,[12],[1.0])...]	(6694,[12,18,28,4...]	2
116	french	[(6694,[936],[1.0])...]	(6694,[60,85,278,...]	2
138	greek	[(6694,[1],[1.0])...]	(6694,[1,82,97,29...]	2
151	french	[(6694,[449],[1.0])...]	(6694,[0,5,9,23,1...]	1
156	vietnamese	[(6694,[1424],[1.0])...]	(6694,[3,5,34,58,...]	1
174	japanese	[(6694,[108],[1.0])...]	(6694,[0,2,16,25,...]	8
189	greek	[(6694,[1250],[1.0])...]	(6694,[0,1,2,36,6...]	3
209	mexican	[(6694,[95],[1.0])...]	(6694,[0,1,3,4,8,...]	3
212	southern_us	[(6694,[151],[1.0])...]	(6694,[55,151,501...]	2
250	italian	[(6694,[1861],[1.0])...]	(6694,[0,2,7,8,29...]	0
255	italian	[(6694,[127],[1.0])...]	(6694,[0,1,8,36,4...]	3
258	irish	[(6694,[11],[1.0])...]	(6694,[0,9,11,17,...]	6
266	mexican	[(6694,[265],[1.0])...]	(6694,[1,2,12,53,...]	2
301	mexican	[(6694,[935],[1.0])...]	(6694,[1,35,70,91...]	2
314	indian	[(6694,[15],[1.0])...]	(6694,[2,8,15,77,...]	2
335	indian	[(6694,[350],[1.0])...]	(6694,[0,2,24,77,...]	8
338	mexican	[(6694,[16],[1.0])...]	(6694,[15,16,43,1...]	2
414	mexican	[(6694,[1367],[1.0])...]	(6694,[0,26,27,35...]	3

در قسمت آخر بصورت کیفی تفاوت مواد غذایی به کار رفته در هر خوشه را بررسی می کنیم .

از میانگین داده های هر خوشه برای تحلیل آن خوشه استفاده می کنیم . پس ابتدا میانگین داده هارا به دست می آوریم :

```
[ ] # محاسبه میانگین مواد غذایی در هر خوشه
cluster_summary = predictions.groupby("prediction").mean()

# نمایش مقادیر میانگین
cluster_summary.show()
```

prediction	avg(id)	avg(prediction)
1	24525.173501577287	1.0
6	25183.488584474886	6.0
3	24478.545134818287	3.0
5	42756.0	5.0
4	23222.13259668508	4.0
8	25370.061425061424	8.0
2	24749.52187902188	2.0
0	24412.467605633803	0.0
7	41351.0	7.0

با توجه به میانگین های هر خوشه ، می توان تفاوت غذاهای به کار رفته در هر خوشه را مقایسه کرد .

همچنین برای شناسایی ۱۰ ماده غذایی پرکاربرد در هر خوشه، از میانگین مقادیر ویژگی‌ها استفاده می‌کنیم:

```
[ ] import numpy as np
    from pyspark.sql.functions import col

    top_features_per_cluster = {}
    for cluster_id in range(model.getK()):
        # فیلتر داده‌های مربوط به هر خوشه
        cluster_data = predictions.filter(col("prediction") == cluster_id)

        # استخراج بردارهای ترکیبی
        cluster_vectors = cluster_data.select("combined_vector").rdd \
            .map(lambda row: row["combined_vector"].toArray()).collect()

        # تبدیل داده‌ها به numpy array و محاسبه میانگین
        cluster_vectors_np = np.array(cluster_vectors)
        avg_features = cluster_vectors_np.mean(axis=0)

        # انتخاب 10 ویژگی برتر
        top_features = sorted(enumerate(avg_features), key=lambda x: x[1], reverse=True)[:10]
        top_features_per_cluster[cluster_id] = top_features

    # نمایش 10 ویژگی برتر برای هر خوشه
    for cluster_id, features in top_features_per_cluster.items():
        print(f"Top 10 features for cluster {cluster_id}: {features}")
```

که نمونه ای از نتیجه ی به دست آمده به صورت زیر است:

```
Top 10 features for cluster 0: [(7, 1.002808988764045), (0, 0.4438202247191011), (2, 0.18258426966292135), (4, 0.15730337078651685), (1, 0.1544943820224719), (9, 0.15168539325842698), (8, 0.15168539325842698), (3, 0.15168539325842698), (6, 0.15168539325842698), (5, 0.15168539325842698), (10, 0.15168539325842698)]
Top 10 features for cluster 1: [(5, 0.8296529968454258), (0, 0.6309148264984227), (9, 0.5362776025236593), (17, 0.43217665615141954), (19, 0.4195583596214511), (34, 0.29652996845425866), (3, 0.29652996845425866), (6, 0.29652996845425866), (8, 0.29652996845425866), (10, 0.29652996845425866), (11, 0.29652996845425866)]
Top 10 features for cluster 2: [(1, 0.23037323037323038), (6, 0.15444015444015444), (3, 0.14671814671814673), (2, 0.13835263835263836), (14, 0.13449163449163448), (4, 0.1274131274131274), (12, 0.1274131274131274), (13, 0.1274131274131274), (15, 0.1274131274131274), (16, 0.1274131274131274), (18, 0.1274131274131274)]
Top 10 features for cluster 3: [(0, 1.0), (1, 0.369284876905041), (10, 0.25791324736225085), (6, 0.24267291910902697), (4, 0.17584994138335286), (8, 0.1735052754982415), (3, 0.1688159437), (11, 0.1688159437), (12, 0.1688159437), (13, 0.1688159437), (14, 0.1688159437)]
Top 10 features for cluster 4: [(13, 0.8397790055248618), (31, 0.4171270718232044), (4, 0.35359116022099446), (5, 0.3425414364640884), (3, 0.30939226519337015), (0, 0.27624309392265195), (1, 0.27624309392265195), (2, 0.27624309392265195), (6, 0.27624309392265195), (7, 0.27624309392265195)]
Top 10 features for cluster 5: [(0, 1.0), (6, 1.0), (38, 1.0), (531, 1.0), (970, 1.0), (1308, 1.0), (1, 0.0), (2, 0.0), (3, 0.0), (4, 0.0)]
Top 10 features for cluster 6: [(0, 0.8584474885844748), (9, 0.6484018264840182), (12, 0.5753424657534246), (23, 0.4292237442922374), (11, 0.3150684931506849), (34, 0.2374429223744292), (13, 0.2374429223744292), (14, 0.2374429223744292), (15, 0.2374429223744292), (16, 0.2374429223744292)]
Top 10 features for cluster 7: [(5, 1.0), (61, 1.0), (131, 1.0), (148, 1.0), (233, 1.0), (376, 1.0), (465, 1.0), (769, 1.0), (945, 1.0), (1411, 1.0)]
Top 10 features for cluster 8: [(2, 0.8624078624078624), (0, 0.8132678132678133), (4, 0.5331695331695332), (3, 0.40540540540540543), (16, 0.2285012285012285), (87, 0.20638820638820637), (17, 0.20638820638820637), (18, 0.20638820638820637), (19, 0.20638820638820637), (20, 0.20638820638820637)]
```

به عنوان مثال دوتایی (7,1.002808988764045) به این معنی است که ماده ی غذایی با ایندکس ۷ در بردار combined_vector با میانگین 1.002808988764045 بیشترین کاربرد را دارد.

برای تحلیل مقایسه‌ای خوشه‌ها با یکدیگر، میانگین ویژگی‌های یک خوشه خاص را با میانگین سایر خوشه‌ها مقایسه می‌کنیم:

```
[ ] import numpy as np
    from pyspark.sql.functions import col

    # شماره خوشه مورد نظر
    cluster_id = 0

    # محاسبه میانگین ویژگی‌ها برای خوشه‌ی خاص
    cluster_data = predictions.filter(col("prediction") == cluster_id)
    cluster_vectors = cluster_data.select("combined_vector").rdd \
        .map(lambda row: row["combined_vector"].toArray()).collect()
    cluster_avg = np.mean(np.array(cluster_vectors), axis=0)

    # محاسبه میانگین ویژگی‌ها برای کل داده‌ها
    overall_vectors = predictions.select("combined_vector").rdd \
        .map(lambda row: row["combined_vector"].toArray()).collect()
    overall_avg = np.mean(np.array(overall_vectors), axis=0)

    # محاسبه تفاوت‌ها
    differences = cluster_avg - overall_avg

    # نمایش تفاوت‌ها
    print(f"Differences for cluster {cluster_id}: {differences}")
```

➡ Differences for cluster 0: [-0.00491729 -0.05101783 -0.01203535 ... 0. 0.
0.]

همینطور که قابل مشاهده است تفاوت خوشه‌ها به صورت تفاوت میانگین نمایش داده می‌شود.